

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: # Section C - Mini Capstone Project
# Food Delivery Analytics Console - Full EDA Repor

np.random.seed(30)

#Generating dataset
restaurants = ["Spice Hub", "Urban Bites", "Tasty Corner", "Food Street", "Royal
cities = ["Vadodara", "Ahmedabad", "Surat"]
cuisines = ["Indian", "Chinese", "Fast Food"]

df = pd.DataFrame({"restaurant_name": np.random.choice(restaurants, 250), "city":
"order_value": np.random.uniform(150, 1000, 250), "delivery_time_mins": np.random
"rating": np.random.uniform(1.0, 5.0, 250), "cuisine_type": np.random.choice(cuis
```

```
In [3]: df
```

```
Out[3]:
```

	restaurant_name	city	order_value	delivery_time_mins	rating	cuisine_ty
0	Royal Kitchen	Vadodara	744.871000	45.258617	2.312274	Ind
1	Royal Kitchen	Surat	807.332157	33.428260	3.644407	Chine
2	Tasty Corner	Vadodara	474.007802	21.751405	2.552070	Chine
3	Food Street	Surat	176.547952	28.172788	3.926836	Ind
4	Urban Bites	Ahmedabad	693.793930	35.976107	3.502453	Fast Fo
...	...	...	...	...	...	...
245	Tasty Corner	Vadodara	860.039367	38.158094	2.394128	Fast Fo
246	Tasty Corner	Vadodara	823.623076	27.472385	1.020470	Ind
247	Urban Bites	Vadodara	518.487789	17.651599	1.243640	Ind
248	Urban Bites	Ahmedabad	566.354380	31.607754	1.440648	Fast Fo
249	Tasty Corner	Ahmedabad	190.826481	43.635083	3.785232	Ind

250 rows × 6 columns



```
In [5]: # Creating a few missing numeric values for the required null-handling step

for column in ["order_value", "delivery_time_mins", "rating"]:
    missing_rows = np.random.choice(df.index, size=5, replace=False)
    df.loc[missing_rows, column] = np.nan

numeric_columns = df.select_dtypes(include=np.number).columns
for column in numeric_columns:
    df[column] = df[column].fillna(df[column].median())
```

```
In [15]: def summary_statistics():
    print("\n--- Summary Statistics ---")
    print(f"Mean order value: Rs {np.mean(df['order_value']):.2f}")
    print(f"Mean delivery time: {np.mean(df['delivery_time_mins']):.2f} minutes")
    print(f"Mean rating: {np.mean(df['rating']):.2f}")
    print(f"Delivery time standard deviation: {np.std(df['delivery_time_mins']):
```

```
In [25]: def distribution_analysis():plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(df["order_value"], bins=15)
plt.title("Order Value Distribution")
plt.xlabel("Order Value")
plt.ylabel("Frequency")
plt.subplot(1, 2, 2)
plt.hist(df["delivery_time_mins"], bins=15)
plt.title("Delivery Time Distribution")
plt.xlabel("Delivery Time")
plt.ylabel("Frequency")
plt.tight_layout()
```



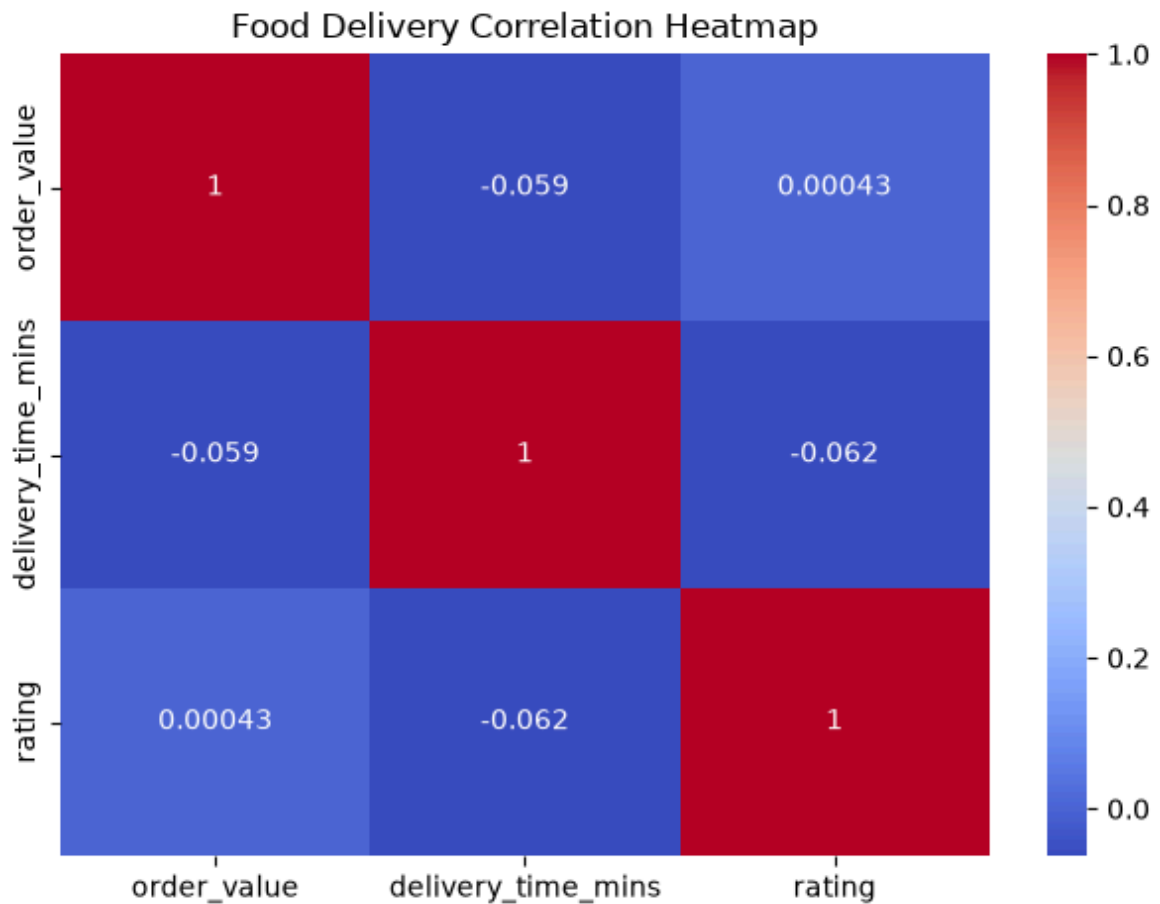
```
In [27]: plt.show()
```

```
In [28]: plt.savefig("distribution_analysis.png")
    print("Graph saved as distribution_analysis.png")
```

Graph saved as distribution_analysis.png
<Figure size 640x480 with 0 Axes>

```
In [31]: def correlation_heatmap():plt.figure(figsize=(8, 6))
    sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap="coolwarm")
    plt.title("Food Delivery Correlation Heatmap")
```

```
plt.tight_layout()
plt.savefig("correlation_heatmap_capstone.png")
plt.show()
```



```
In [33]: def restaurant_performance():

    result = df.groupby("restaurant_name")[["order_value", "delivery_time_mins",

    print("\n--- Restaurant Performance Report ---")
    print(result)
```

```
In [ ]: def final_report():

    performance = (
        df.groupby("restaurant_name")["rating"]
        .mean()
        .sort_values(ascending=False)
        .head(3)
    )

    corr = df.select_dtypes(include=np.number).corr().abs()
    corr = corr.where(corr != 1)

    strongest_pair = corr.stack().idxmax()
    strongest_value = corr.stack().max()

    mean_delivery = df["delivery_time_mins"].mean()
    std_delivery = df["delivery_time_mins"].std()

    print("\n===== FINAL REPORT =====")
```

```

print("\nTop 3 restaurants:")
for i, (restaurant, rating) in enumerate(performance.items(), 1):
    print(f"{i}. {restaurant}: {rating:.2f}")

print("\nStrongest correlation:")
print(f"{strongest_pair[0]} vs {strongest_pair[1]}: {strongest_value:.2f}")

print(f"\nAverage delivery time: {mean_delivery:.2f} minutes")
print(f"Delivery time standard deviation: {std_delivery:.2f} minutes")

while True:

    print("\n===== FOOD DELIVERY ANALYTICS CONSOLE =====")
    print("1. Summary Statistics")
    print("2. Distribution Analysis")
    print("3. Correlation Heatmap")
    print("4. Restaurant Performance Report")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        summary_statistics()

    elif choice == "2":
        distribution_analysis()

    elif choice == "3":
        correlation_heatmap()

    elif choice == "4":
        restaurant_performance()

    elif choice == "5":
        final_report()
        print("\nProgram ended.")
        break

    else:
        print("Please enter a valid choice.")

```

===== FOOD DELIVERY ANALYTICS CONSOLE =====

```

1. Summary Statistics
2. Distribution Analysis
3. Correlation Heatmap
4. Restaurant Performance Report
5. Exit
--- Summary Statistics ---
Mean order value: Rs 567.31
Mean delivery time: 29.86 minutes
Mean rating: 2.88
Delivery time standard deviation: 7.34

```

===== FOOD DELIVERY ANALYTICS CONSOLE =====

```

1. Summary Statistics
2. Distribution Analysis
3. Correlation Heatmap
4. Restaurant Performance Report
5. Exit

```

--- Restaurant Performance Report ---

restaurant_name	order_value	delivery_time_mins	rating
Food Street	587.577228	28.853148	2.904346
Royal Kitchen	602.590788	30.198464	3.018801
Spice Hub	557.034588	28.478380	2.780605
Tasty Corner	594.014682	30.739303	2.720734
Urban Bites	496.599496	30.910050	2.944741

===== FOOD DELIVERY ANALYTICS CONSOLE =====

1. Summary Statistics
2. Distribution Analysis
3. Correlation Heatmap
4. Restaurant Performance Report
5. Exit

===== FOOD DELIVERY ANALYTICS CONSOLE =====

1. Summary Statistics
2. Distribution Analysis
3. Correlation Heatmap
4. Restaurant Performance Report
5. Exit

In []: